CHAPTER 5

RESEARCH ARCHITECTURE AND TRUST-CENTRIC CONSENT MODEL

5.1 Introduction

In this chapter, firstly the research architecture on which the whole framework is built is presented. There are three layers in the architecture, and the role of each layer is explained in this chapter. Furthermore, a model is proposed to represent patient consent as an evaluable authorisation object; this provides a complete solution to research objective 1. In this chapter, we also present the results of the validation of the proposed solution to research question one. To validate this research question, we built the HALAH prototype and used the following technologies to test the system:

1. **Blockchain:** A blockchain is a decentralised and distributed ledger that records transactions across many computers so that a recorded entry cannot be altered retroactively without altering the blocks that follow it (Wood 2014). In this research, the Ethereum test network Sepolia (Ethereum Foundation 2026b) is used to anchor the state of an approved consent as a soulbound token (Cai 2022) and to record its revocation.
2. **Solidity:** The consent contract is written in Solidity, the contract language of Ethereum (Ethereum Foundation 2026a), and was deployed to Sepolia with the Remix environment (Ethereum Foundation 2026c). The contract stores the start time, the expiry time and the revocation flag of each consent and reports whether the consent is valid at a given time.
3. **Python and Streamlit:** The Controller and the four role interfaces of the prototype, for the patient, the guardian, the doctor and the auditor, are written in Python. The interfaces use the Streamlit library, and the application talks to Sepolia through the web3 library. Consents, approvals, decisions and the audit ledger are kept in a local SQLite database.
4. **EIP-712 and Groth16:** Each guardian approval is a typed-data signature under EIP-712 (Bloemen, Logvinov & Evans 2017), so an approval is bound to one consent, one guardian and one network. The zero-knowledge proof that a consent has not expired is written in circom (iden3 2026b) and is generated and verified with snarkjs (iden3 2026a) under the Groth16 scheme (Groth 2016).
5. **Synthea:** The healthcare records of the prototype are synthetic patients produced by the Synthea generator (Walonoski et al. 2018). The dataset is described in Section 5.7.

The outline of this chapter is as follows. In Section 5.2, we outline the three layers of the architecture and their components. Subsequently, in Section 5.3 we explain where trust is evaluated and where the boundaries between the layers lie, and in Section 5.4 we map the architecture to the six research objectives. In Section 5.5, we present the trust-centric consent model: the creation workflow, the structure of the consent object C = (I, A, P, T, L, E), and the creation procedure as Algorithm 1. Furthermore, in Section 5.6, we discuss the aims of engineering the prototype system used for validating the proposed consent model. In Section 5.7, we outline and explain the dataset used for validation purposes. In Section 5.8 we explain in detail in a stepwise manner the working of the prototype for consent creation. Finally, in Section 5.9 we discuss the results obtained from the

validation of the consent model and the evaluation of the architecture, and Section 5.10 concludes the chapter.

5.2 Layered Architecture and Component Decomposition

The proposed research architecture adopts a Model–View–Controller (MVC) design to provide a structured implementation foundation for the trusted multi-party consent management framework. In the proposed framework, MVC does more than separate data, interfaces, and application logic. It is extended to support a separation between the representation of consent, the interaction of independent stakeholders, and the enforcement of security-critical authorisation decisions. A healthcare consent decision may involve patients, guardians, healthcare professionals, and auditors, each of whom may possess different identities, authorities, and responsibilities. Consequently, the system cannot rely on a single user authentication event or a static permission entry to determine whether access should be granted. Instead, the authorisation decision is the result of evaluating independently verifiable security conditions together.

At the highest level, the architecture consists of three complementary layers. The View Layer represents the interaction boundary with independent healthcare stakeholders. The Controller Layer represents the security decision and coordination boundary. The Model Layer represents the persistent trust, consent, policy, and authorisation state. Figure 5.1 illustrates the conceptual relationship among these layers. The protected healthcare data are deliberately positioned outside the three MVC layers as a protected resource whose access is mediated by the Controller rather than directly exposed to the View.

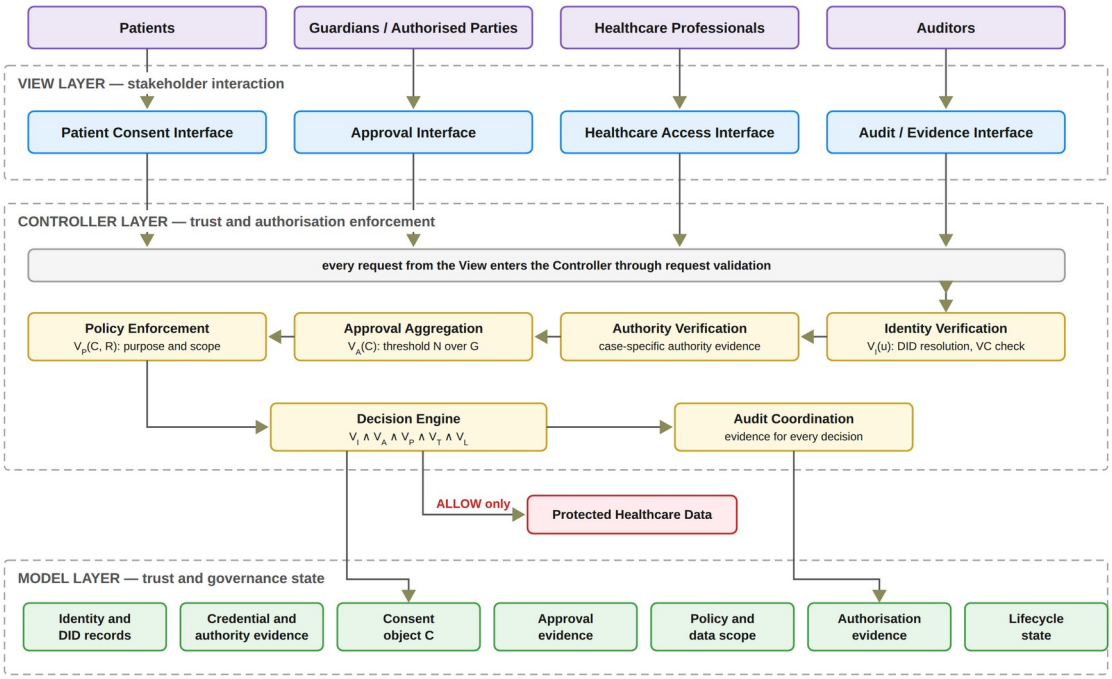


Figure 5.1: Layered MVC architecture and component decomposition of the HALAH framework.

An action submitted through the View represents a request or claim rather than a trusted authorisation outcome. The Controller Layer constitutes the principal security enforcement boundary. It receives requests from the View and coordinates the evaluation of the relevant trust conditions. These include decentralised identity verification, credential and authority verification, multi-party approval evaluation,

policy and data-scope evaluation, and final authorisation. The Controller also generates audit evidence associated with the resulting decision. Consequently, the Controller acts as the coordination point through which sensitive operations must pass before protected healthcare data can be accessed. The Model therefore provides the information required for the Controller to evaluate an access request, but the existence of a Model object does not itself imply that access is permitted. MVC was selected in preference to a purely on-chain design for two reasons. First, the authorisation decision must combine the identity, threshold, policy, temporal, and lifecycle evaluation performed by the Controller with the anchoring of consent state on the blockchain. Second, each stakeholder interface can be replaced without altering the enforcement logic, so the trust predicates are evaluated independently of any particular user interface.

Figure 5.2 presents this decomposition. The View is decomposed according to the principal stakeholder roles. The patient interface supports consent specification and consent-related actions. The guardian or other authorised-participant interface supports independent approval. The healthcare professional interface supports authorised data-access requests. The auditor interface supports inspection of authorisation evidence and audit history. These interfaces represent different interaction responsibilities but share the same architectural principle: none of them independently determines whether access should be granted. The Controller is decomposed into six principal security functions. Identity Verification establishes whether a participant can be identified through the relevant decentralised identity mechanism. Authority Verification evaluates whether the participant possesses appropriate evidence concerning the required role or authority. Approval Aggregation determines whether independently verified approvals satisfy the predefined threshold. Policy Enforcement evaluates whether the requested purpose and data scope comply with the consent policy. Decision Engine combines the resulting predicates into the final authorisation decision. Finally, Audit Coordination records the relevant evidence and decision events. The Model is decomposed into seven principal state representations: DID, VC, Consent, Approval, Policy, Evidence, and Lifecycle. This decomposition establishes an explicit distinction between state, interaction, and enforcement.

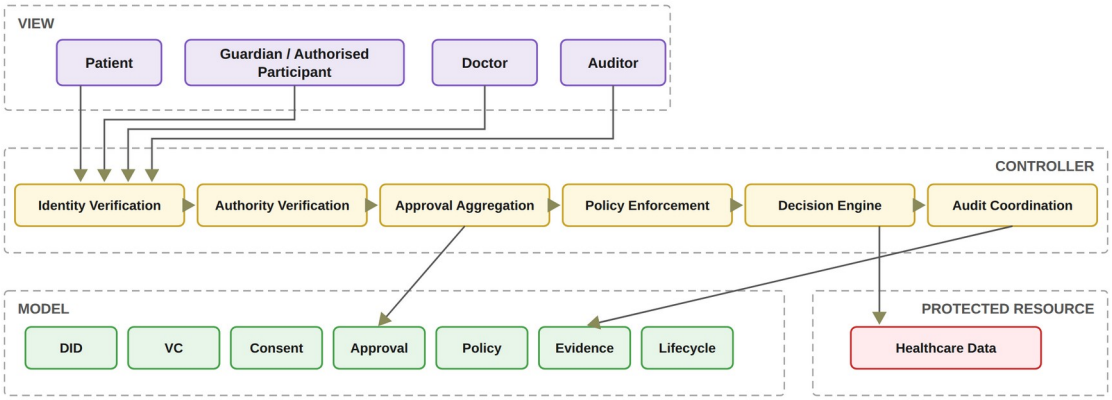


Figure 5.2: Functional decomposition of the MVC components.

In the implemented prototype, the Controller comprises eight principal functions rather than six, since a zero-knowledge (ZK) Proof Service generates and verifies the Groth16 proof that feeds the Authorisation Decision, and an On-chain Anchor Service anchors the Sepolia mint as a background process. A revoke transaction is sent when the patient revokes a consent, and the patient view waits for its confirmation. The Model correspondingly contains the ConsentSBTv3 soulbound contract deployed on Sepolia together with the application audit ledger, both of which an independent verification tool reads, as described in Chapter 10. In the present prototype, each DID must be well formed, and the doctor and each guardian must hold a valid verifiable credential from a

demo issuer. The guardian's credential is checked before the guardian signs, and V_I combines the doctor's credential check with the EIP-712 verification of each listed guardian's signature. The requester's DID and jurisdiction are declared in the interface rather than authenticated, and wallet-held signing keys, full DID resolution and proof that the user controls the DID are deferred to the next stage. However, each guardian approval is an EIP-712 typed-data signature bound to the consent identifier, so the multi-party threshold is enforced over signed approvals rather than over interface actions.

5.3 Trust Evaluation and Architectural Boundaries

The architectural layers cooperate through a controlled authorisation sequence rather than through unrestricted direct communication. A healthcare data-access request is not evaluated according to a single authentication event. As illustrated in Figure 5.3, the Controller coordinates a sequence of trust predicates representing the conditions established by the sub-research questions. The first condition, $V_I(u)$, determines whether the requesting participant possesses valid identity and authority evidence. The second condition, $V_A(C)$, determines whether the required independently verified participants have collectively satisfied the authorisation threshold N. The third condition, $V_P(C, R)$, determines whether the requested operation complies with the consent policy, including purpose and data-scope restrictions. The fourth condition, $V_T(C, t)$, determines whether the consent remains valid at the time of the request. The fifth condition, $V_L(C)$, determines whether the consent remains in a valid lifecycle state. The integrated decision can therefore be expressed as Equation (3.2). In the current prototype, the Consent Engine derives the lifecycle state, a request proceeds only when this state is ACTIVE, and the revocation flag and the token are also checked within V_A. The prototype interface also shows a check labelled VL, which compares the jurisdiction declared in the request with the jurisdiction recorded in the consent. This thesis denotes that check V_J and treats jurisdiction as a contextual policy condition for the next stage (Chapter 8).

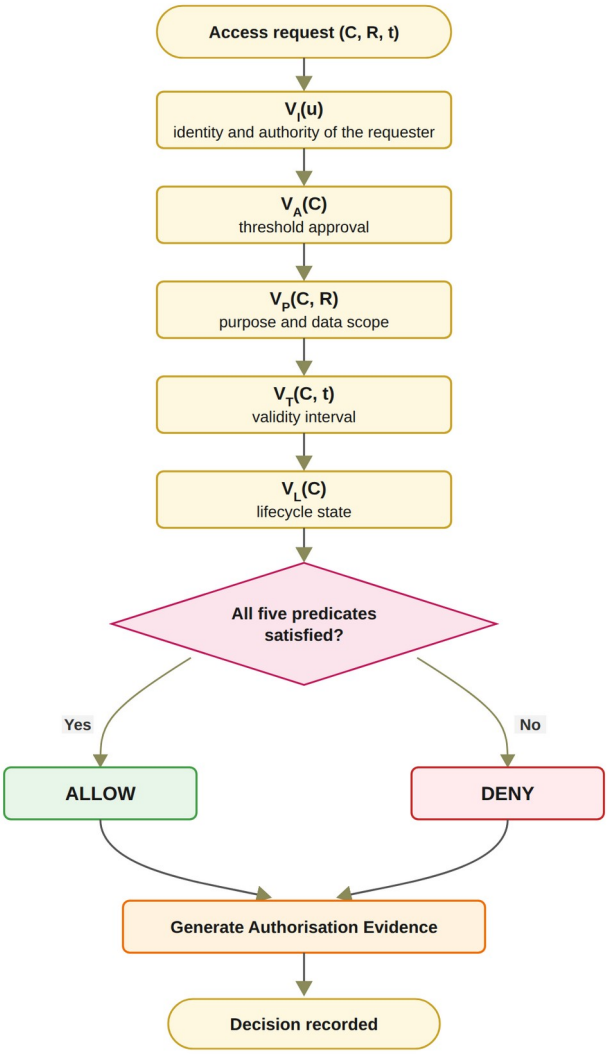


Figure 5.3: Trust predicate evaluation and authorisation decision flow (in the current prototype, V_I also checks the EIP-712 signature of each listed guardian, and a refusal for a consent that is not ACTIVE is recorded in the audit ledger like any other refusal, while the check labelled VL in the prototype interface is the exploratory jurisdiction check V_J).

If any required predicate evaluates to false, the decision is DENY. The Controller does not itself create these trust conditions. Instead, it evaluates evidence represented by the Model and coordinates the resulting decision. For the interpretation of the architecture: the Controller is a decision-enforcement mechanism, whereas the Model is a representation of trust and governance state. Both successful and unsuccessful decisions are subsequently associated with authorisation evidence, which supports accountability and subsequent verification. The layered architecture also establishes explicit trust boundaries. The View is considered an interaction boundary and therefore cannot be treated as a trusted source of authorisation decisions. The Controller represents the security enforcement boundary. It is responsible for applying the authorisation rules consistently and for preventing direct access to protected healthcare data when the required trust conditions are not satisfied. The Model represents the security-state boundary. Its records provide the evidence and state required for authorisation evaluation, but stored state alone does not constitute permission. In particular, the presence of a consent record, credential, or approval record cannot independently produce an access decision. The protected healthcare data represent the final resource boundary, and access is permitted only through the authorisation path enforced by the

Controller. The proposed framework does not claim that MVC itself provides security. Instead, security arises from the explicit placement of trust-sensitive operations within the Controller, the structured representation of security state within the Model, and the prevention of direct security decisions at the View layer. In the present prototype, the Controller is a single trusted server that both evaluates the predicates and writes the audit ledger, and accordingly, the on-chain anchoring of consent state and the independent verification tool provide the external evidence against which the Controller's own records can be checked. Figure 5.4 puts the same conditions in the order a request meets them. A DID answers who is asking, a verifiable credential and a signed approval answer what that participant may do, the threshold answers whether the members of G agreed, and the policy, temporal and lifecycle checks with the zero-knowledge proof answer whether the consent is still valid for this request. The figure also states, gate by gate, what the current prototype checks, so the reader can see where the design and the prototype differ before the detailed sections.

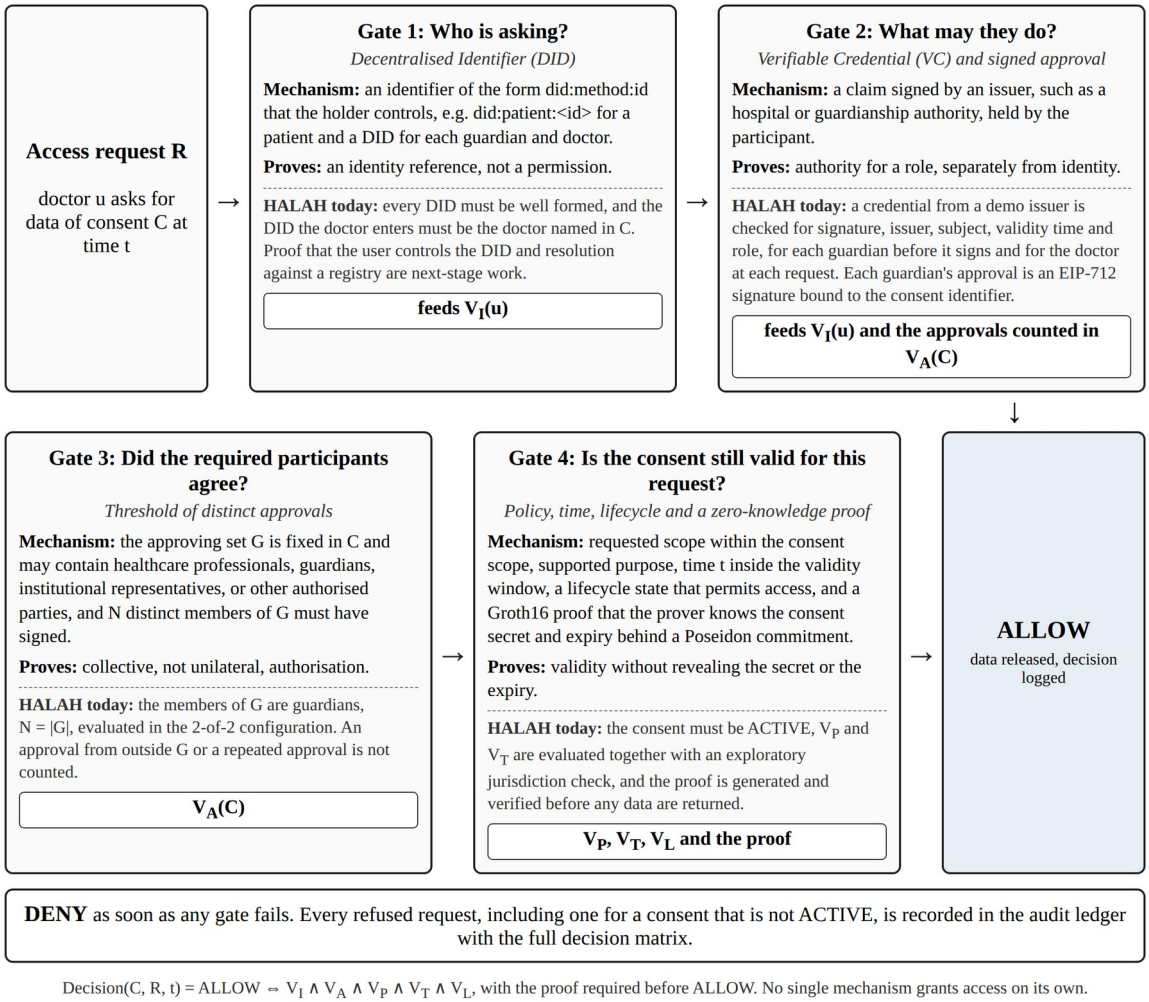


Figure 5.4: The four gates that an access request passes before data are released: identity (DID), authority (VC and signed approval), collective approval (threshold), and validity of the consent for the request (policy, time, lifecycle and the zero-knowledge proof). Each gate names what the current prototype checks, and a failure at any gate produces DENY. The prototype records the decision matrix in the audit ledger for every request, including one against a consent that is not ACTIVE.

5.4 MVC Architecture and the Research Objectives

The MVC architecture also provides a direct implementation structure for the research objectives, as summarised in Table 5.1. RO1 is primarily represented in the Model through the development of the trust-aware consent object $C = (I, A, P, T, L, E)$. RO2 is implemented through the identity and authority verification mechanisms coordinated by the Controller. RO3 is realised through the multi-party approval and threshold evaluation process. RO4 is implemented through the policy evaluation mechanisms that constrain the authorised purpose and healthcare data scope. For RO5, temporal conditions and lifecycle state are represented in the Model, while V_T and V_L are evaluated by the Controller before an access decision is produced. RO6 operates across the complete architecture rather than being confined to a single MVC layer. Formal security assurance evaluates the logical properties of the integrated model, while empirical validation evaluates whether the implementation consistently realises those properties. The evidence generated by the Controller and stored in the Model is what the empirical validation and the independent verifier examine.

Table 5.1: Mapping between research objectives and MVC components

Research Objective	Primary MVC Component	Core Mechanism	Expected Contribution
RO1	Model	Trust-aware consent representation	Transforms consent from a passive permission record into a structured and evaluable authorisation object.
RO2	Controller and Model	DID and VC verification	Establishes trustworthy participants while explicitly separating identity from authorisation authority.
RO3	Controller	Verified multi-party threshold evaluation	Establishes collective trust and prevents unilateral authorisation when multiple approvals are required.
RO4	Controller and Model	Policy, purpose, and data-scope evaluation	Ensures that valid participants and sufficient approvals remain constrained by the authorised purpose and healthcare data scope.
RO5	Model and Controller	Temporal validity and consent lifecycle enforcement	Ensures that authorisation remains valid only within its permitted time interval and lifecycle state, and that expiration, revocation, and unauthorised state transitions are correctly enforced.
RO6	Model, View, and Controller	Formal security analysis and empirical validation	Will provide formal assurance (next stage) of the defined trust and authorisation properties and empirically evaluate whether the implemented framework correctly enforces authorised and unauthorised healthcare data-access decisions.

Table 5.2 summarises how the proposed framework differs from traditional EHR access control.

Table 5.2: Traditional EHR access control compared with the HALAH framework

Aspect	**Traditional EHR access control**	**HALAH framework**
Identity	Central identity provider; institution login	A DID for every participant; identity kept separate from authority
Consent	A stored record that permission was given	An object C = (I, A, P, T, L, E), checked at every request
Approval	A static permission set by one party	All listed guardians sign (EIP-712); no single party can approve alone
Access check	Log in, then check a static permission	VI ∧ VA ∧ VP ∧ VT ∧ VL, plus a Groth16 proof before data is released
Time and revocation	Hard to verify once the permission is stored	Validity window and revocation checked each time; on-chain revoke
Audit	A record that consent occurred	Every access decision, allowed or denied, logged off-chain with its decision matrix; consent state anchored on Sepolia

The contribution of the proposed architecture is the way MVC is used to carry a trust-oriented authorisation model, not the use of MVC itself. This design transforms multi-party consent management from a conventional record-management problem into a distributed security-governance problem. The evaluable consent object as such is already present in the literature. The key innovation is that consent effectiveness depends on the joint satisfaction of identity, authority, collective approval, policy, temporal, and lifecycle conditions on every access decision. Blockchain or other tamper-resistant storage mechanisms are integrated as infrastructure for preserving selected authorisation evidence. However, the blockchain itself is not treated as the source of trust. Its role is to strengthen integrity, non-repudiation, and auditability of the evidence generated by the authorisation process. The detailed solutions for RQ1 to RQ6 are presented in Chapters 5 to 10, and the formal assurance of the complete decision model remains for the next stage. The next section presents the solution for RO1, the consent object on which the other objectives build.

5.5 Trust-Centric Consent Model

This section presents the solution for RO1. Section 5.5.1 shows the consent creation workflow, Section 5.5.2 defines the structure of the consent object, Section 5.5.3 presents the creation procedure and the algorithm, Section 5.6 describes the aims of the prototype in which the model is realised, Section 5.7 the dataset, Section 5.8 the steps of consent creation, and Section 5.9 validates and evaluates it.

RO1 addresses RQ1 by developing a trust-aware consent model that represents patient consent as a structured and evaluable authorisation object. The purpose of this objective is to establish a formal representation of what the patient intends to authorise, who is permitted to participate in the authorisation process, which healthcare data may be accessed, and under what conditions. The proposed consent object is represented as Equation (5.1).

$$C = (I, A, P, T, L, E) \tag{5.1}$$

In Equation (5.1), I represents patient and participant identity information, A represents authorisation requirements, P represents policy constraints, and E represents authorisation evidence. T represents the temporal validity window, and L represents the current state of the consent object and its permissible transitions. The temporal component bounds the consent in time, and the lifecycle component states whether the consent can currently be used. The identity component identifies the patient and relevant participants. The authorisation component defines the required participants and

approval threshold. The policy component specifies the permitted purpose and healthcare data scope. The evidence component records the information required to support subsequent verification and audit. The consent model therefore separates the specification of consent from the final access decision. A consent object does not automatically imply that every subsequent request is authorised. Instead, the object provides the conditions against which a request can be evaluated.

5.5.1 Consent Creation Workflow

Figure 5.5 describes the creation of a trust-aware consent object. The process begins with a patient request and ends with a structured consent record that can be verified by subsequent authorisation services. The workflow first collects the patient identity and the intended healthcare participants. It then specifies the purpose and data scope of the consent. In the HALAH implementation, the patient interface allows the patient to select a patient DID, specify an authorised doctor, identify guardian DIDs, and select healthcare data modules such as Observation, Medication, Condition, and Procedure. The implementation also collects the start and expiry time of the consent. In the current interface, the purpose is fixed to Treatment and is not selected by the patient, and the purpose vocabulary check is therefore exercised by the automated tests rather than by the interface.

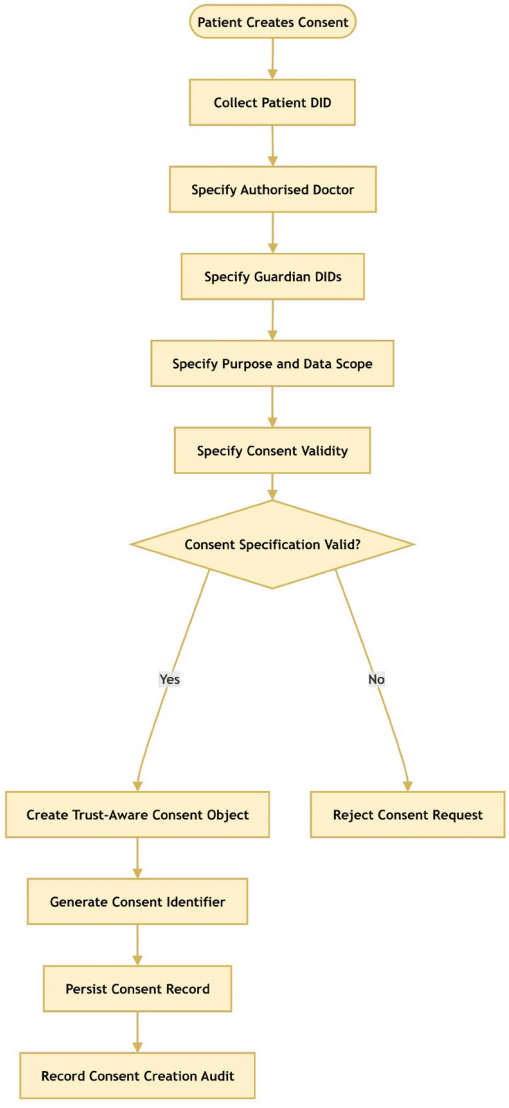


Figure 5.5: RO1 workflow for trust-centric consent creation.

The specification is accepted only when the required fields are valid. For example, the expiry time must be later than the start time, the data scope must not be empty, and each guardian DID must be well formed. After validation, the system creates the consent object, assigns a consent identifier, persists the record, and records the creation event in the audit layer. This process ensures that the consent object contains sufficient information for subsequent identity verification, multi-party authorisation, policy evaluation, and auditability. Once the consent object is created, its lifecycle state L becomes PENDING, because the consent cannot produce an effective authorisation decision until the required approvals have been obtained. In the current prototype, this state is PENDING_SIGNATURE, and the interface also records a jurisdiction code, AU in the demonstrated transactions, for the exploratory jurisdiction check (Chapter 8).

5.5.2 Structure of the Consent Object

Figure 5.6 shows how the validated information is combined to construct the consent object. The resulting object therefore captures the patient's preference and the evidence needed for later authorisation decisions.

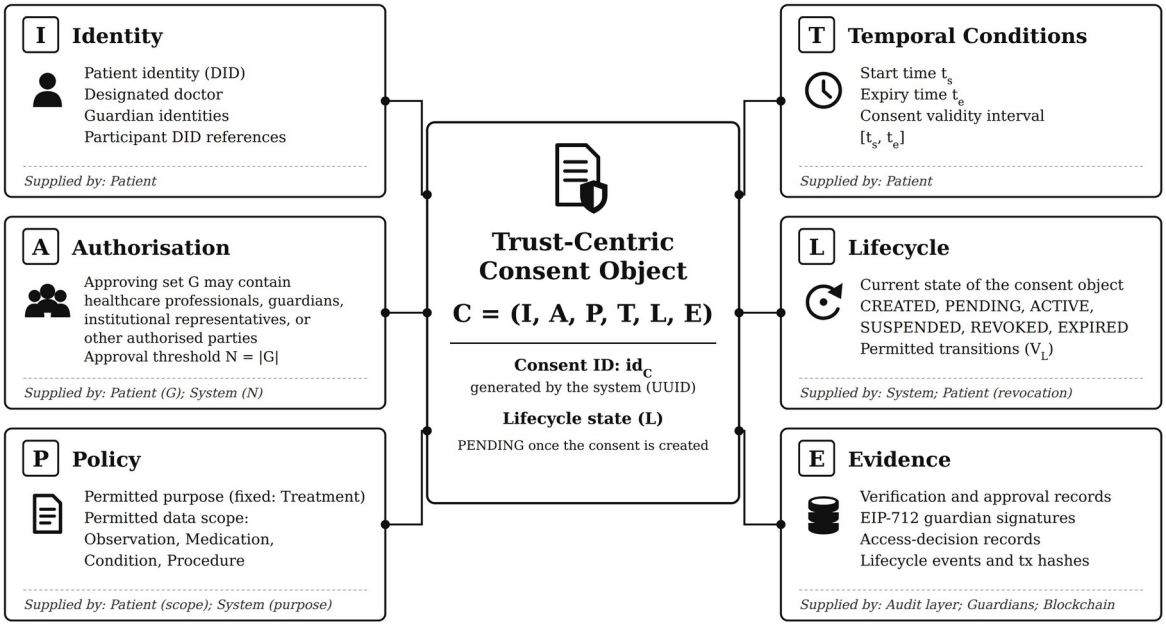


Figure 5.6: Six-component consent model C = (I, A, P, T, L, E) with the content and supplier of each component, where L is the lifecycle state. The states shown are those of the design, and the states implemented in the prototype are given in Chapter 9.

Unlike the consent objects reported in the literature, this object carries the guardian set and threshold together with the purpose, data scope, validity interval, and lifecycle state.

5.5.3 Trust-Aware Consent Creation Procedure and Algorithm

The Trust-Aware Consent Creation procedure establishes a valid and auditable consent object by sequentially validating the participating entities, consent purpose, data scope, and temporal constraints before the consent is persisted. The procedure consists of the following steps.

Step 1 – Validate Patient Identity. The system first validates the patient's identity Ip. The identity is checked against the available identity and credential verification mechanisms to ensure that the consent request originates from a recognised patient entity. If Ip cannot be successfully validated, the procedure terminates immediately and returns REJECT.

Step 2 – Validate the Authorised Doctor. The system validates the identity and authorisation status of the designated doctor D. This step determines whether the doctor is a recognised healthcare participant and whether the presented credentials are sufficient for participation in the requested consent relationship. An invalid or unauthorised doctor results in rejection of the consent request.
Step 3 – Validate Guardian or Proxy Participants. If a guardian or proxy is specified, the system validates the guardian set G. Each guardian is checked to determine whether the individual or entity possesses the required authority to participate in consent-related decisions. This step prevents an arbitrary third party from being treated as an authorised representative of the patient.
Step 4 – Validate the Intended Purpose. The intended data-use purpose Pu is validated to ensure that the purpose is explicitly specified and conforms to the supported healthcare-use policies. The purpose is subsequently bound to the consent object so that authorisation is not interpreted as unrestricted access to the patient's healthcare information.
Step 5 – Validate the Data Scope. The requested data scope SC is validated to determine precisely which healthcare resources may be accessed under the consent. The scope may represent specific resource categories, such as observations, medications, procedures, or other resource categories. This step establishes the principle of least privilege by ensuring that consent is associated with explicitly defined data resources rather than the patient's entire healthcare record.
Step 6 – Validate the Consent Validity Interval. The system validates the temporal validity interval $T = (ts, te)$, where ts represents the consent start time and te represents the expiration time. The system verifies that the interval is logically valid by requiring $te > ts$. If this condition is not satisfied, the consent request is rejected.
Step 7 – Define the Authorisation Requirement. After all input elements have been validated, the system defines the authorisation requirement A. This requirement specifies the conditions that must subsequently be satisfied before access can be granted. Depending on the system configuration, A may incorporate patient approval, guardian approval, doctor authority, role requirements, multi-party approval, scope restrictions, purpose restrictions, and temporal constraints.
Step 8 – Generate a Unique Consent Identifier. The system generates a unique consent identifier idC. This identifier provides a persistent reference to the consent object and enables subsequent operations, including consent verification, access authorisation, modification, revocation, and audit.
Step 9 – Construct the Consent Object. The validated information is combined to construct the consent object $C = (I, A, P, T, L, E)$, where I represents the verified identity information, A represents the authorisation requirements, P represents the consent purpose and associated permissions, T represents the validity interval, L represents the lifecycle state, and E represents the evidence associated with the consent decision. The resulting object therefore captures not only the patient's preference but also the evidence required to support subsequent authorisation decisions.
Step 10 – Persist the Consent Object. The constructed consent object C is persisted in the consent-management infrastructure. The persistence mechanism provides a stable representation of the consent state that can subsequently be referenced by the access-control and authorisation components.
Step 11 – Record Consent Creation Evidence. Finally, the system records evidence associated with the creation event, including the consent identifier, participating entities, relevant timestamps, and the resulting consent state. This evidence provides an auditable record that can be used to reconstruct and verify the consent lifecycle.
Step 12 – Return the Consent Result. If all validation and construction operations are completed successfully, the system returns the newly created consent object C. If any mandatory validation fails during the process, the system terminates the procedure and returns REJECT.
The overall procedure can therefore be represented as a sequential trust establishment process that $Ip \rightarrow D \rightarrow G \rightarrow Pu \rightarrow SC \rightarrow T \rightarrow A \rightarrow idC \rightarrow C \rightarrow E$.

This sequence ensures that consent is created only after the relevant identity, authority, purpose, scope, and temporal constraints have been validated. Consequently, the resulting consent object serves not merely as a record of patient preference, but as a structured authorisation artefact that can provide evidence for subsequent healthcare data-access decisions.

Algorithm 1 Trust-Aware Consent Creation

Require: Patient identity I_p, authorised doctor D, guardian set G, purpose P_u, data scope S_C, validity interval T

Ensure: Consent object C or rejection

```
Validate I_p
if I_p is invalid then
    return REJECT
end if
Validate authorised doctor D, guardian set G, purpose P_u and data scope S_C
if any of them is invalid then
    return REJECT
end if
Validate validity interval T = (t_s, t_e)
if t_e ≤ t_s then
    return REJECT
end if
Define authorisation requirement A
Generate consent identifier id_C
Construct C = (I, A, P, T, L, E)
Persist C
Record consent creation evidence
return C
```

Algorithm 1 formalises the creation process. The algorithm does not grant data access directly. Instead, it creates a consent object that can later be evaluated by the authorisation mechanism. The consent is placed in the PENDING state (PENDING_SIGNATURE in the prototype), and nothing is anchored on-chain until all guardians approve. The algorithm also establishes the relationship between RO1 and the remaining objectives. RO2 uses the identity and authority information contained in I, RO3 evaluates the authorisation requirements contained in A, and RO4 evaluates the policy constraints contained in P. Consequently, RO1 provides the data structure upon which the subsequent trust-oriented authorisation mechanisms operate.

5.6 Prototype Implementation

RO1 has been completed at the conceptual and prototype level, and the HALAH patient interface creates the consent object, assigns the identifier, and records the creation event in the audit layer. The main contribution of RO1 is that consent is represented as an evaluable authorisation object rather than as a binary permission. In the current prototype, the patient DID is selected from the loaded record set. The doctor DID and each guardian DID must be well formed and must hold a valid credential for its role, issued by a demo issuer, before the consent is stored, as qualified in Chapter 6. The validation evidence for consent creation, including the demonstration scenarios and the unit-level tests, is presented in Section 5.9.

5.6.1 Aims of Engineering the Prototype System

The primary aim of engineering the prototype system was to simulate the working of the HALAH framework, in which patient consent is created as an evaluable object and every access request is evaluated against it. In particular, in this section, our objective was to use the developed HALAH prototype to test the efficacy of the consent model proposed in Section 5.5: a patient must be able to create the object C = (I, A, P, T, L, E) through an interface, and the object must carry every component that the later checks need. For validating the other objectives of this thesis, we have extended the same prototype (see Chapters 6 to 9).

As discussed in Chapter 2, one of the major challenges of consent management in healthcare is that a stored record of permission does not establish who may act on it, whether every required party agreed, or whether a request stays within the purpose and the data scope that the patient set. With our framework, the consent object carries these conditions, and the prototype makes the design testable: the same code that the interface calls is called by the automated tests, so a check that the interface enforces can also be attacked directly at the service level, which is how the gaps closed by the hardening of 1 October 2026 were found. The prototype also produces evidence: every creation, approval, decision and revocation is written to the audit ledger with the result of each check, so the claims of this and the following chapters can be traced to recorded events rather than to screenshots alone.

We used the Ethereum test network Sepolia as the computing platform for the on-chain part of the prototype, and Solidity as its programming language (Ethereum Foundation 2026a). Sepolia was chosen because it runs the same virtual machine as the Ethereum main network, so the contract and its transactions behave as they would in production, while its ether has no monetary value, which allowed the mint and revoke transactions to be repeated during development. The prototype is not engineered for throughput or for a production deployment; its purpose is to validate the design.

5.7 Datasets Used for Validation

The prototype uses synthetic patient records from the Synthea 1K sample (Walonoski et al. 2018), taken from synthea.mitre.org in its CSV export rather than its FHIR R4 export (HL7 International 2019). Synthea generates realistic but artificial patients, so no real patient data is involved. The Synthea 1K sample is a public synthetic healthcare dataset of 1,000 simulated patient records, with patient demographics, medical conditions, medications and procedures in CSV files. The prototype loads a subset of patients together with their conditions, observations, medications and procedures, and these tables provide the healthcare data modules that a consent can authorise, such as Observation, Medication and Procedure. The records are used only to construct representative consent workflows and the healthcare data-access scenarios used to test the prototype. The authorisation decisions do not depend on the clinical content, and no medical content is written to the chain. The records are not changed. The only step is the selection of the subset loaded into the local database of the prototype.

Table 5.3 shows the Synthea tables loaded by the prototype, the number of records in the loaded subset, and the consent module that each table supplies. The patient DID of the prototype is derived from the Synthea patient identifier, and the records of a patient are selected by that identifier when a consent authorises the module.

Table 5.3: Synthea tables loaded by the prototype and the consent module supplied by each

Synthea table	Records in the loaded subset	Record selected by	Consent module
patients.csv	113	Id (the source of the patient DID)	Patient identity in the patient interface
observations.csv	4,428	PATIENT	Observation
medications.csv	2,931	PATIENT	Medication
conditions.csv	4,748	PATIENT	Condition
procedures.csv	4,237	PATIENT	Procedure

5.8 Steps in the Working of the Prototype System for Consent Creation

In order to create a consent, the HALAH prototype carries out its work at two levels, namely the patient interface and the service layer. In the patient interface, the consent

specification is entered and checked. In the service layer, the consent object is constructed, stored and recorded in the audit ledger. The steps at both levels follow Algorithm 1 and are explained below in Section 5.8.1 and Section 5.8.2 respectively, and Figure 5.7 shows the patient interface after a consent has been created.

5.8.1 Steps in the patient interface

1. The patient selects the role Patient and their own DID, which the prototype loads from the record set.

2. The patient enters the DID of the authorised doctor and the DIDs of the guardians, separated by commas.

3. The patient selects the data modules from Observation, Medication, Condition and Procedure, and sets the start and expiry times.

4. The patient presses Initial Consent. The prototype checks that at least one guardian is listed, that the doctor DID holds a valid doctor credential, that each guardian DID holds a valid guardian credential, that no guardian appears twice, that the data scope is not empty and that the expiry time is later than the start time. If any check fails, an error message names the failed field and nothing is stored.

5.8.2 Steps in the service layer

5. If every check passes, the service layer assigns the consent identifier, stores the object in the state PENDING_SIGNATURE with N set to the number of guardians, writes the CREATE_CONSENT event to the audit ledger, and returns the identifier to the interface, which shows it to the patient.

6. The new consent appears in the patient's list with 0 of N approvals, where it waits for the guardians, whose steps are presented in Chapter 7.

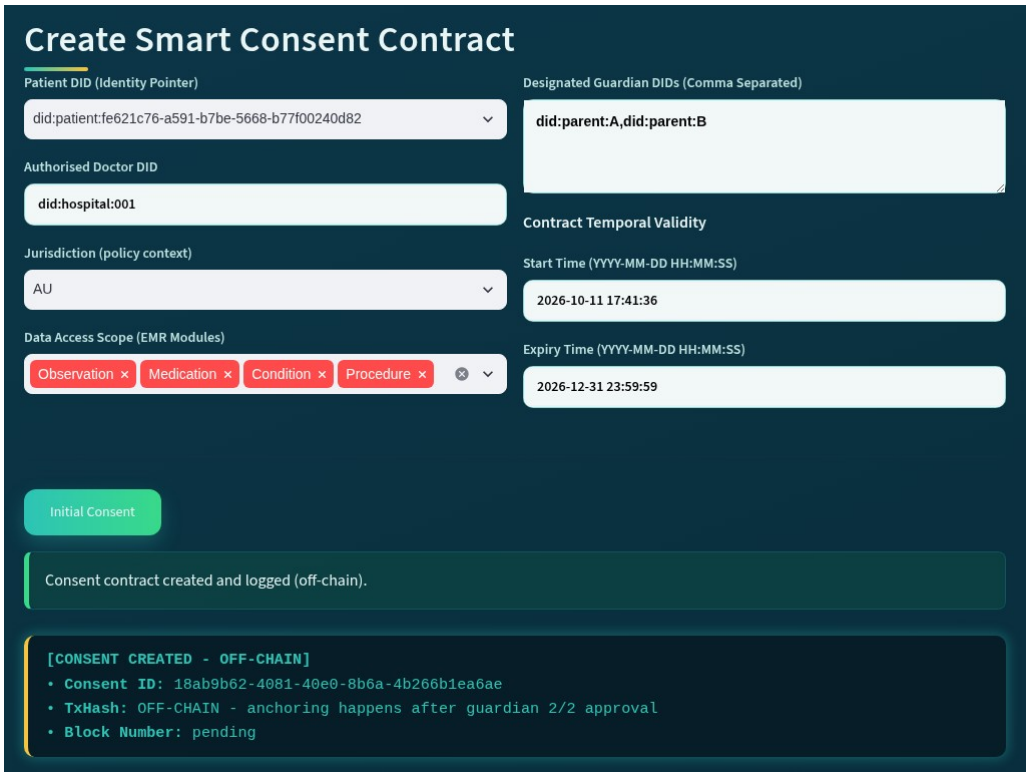


Figure 5.7: The patient interface of the HALAH prototype after a consent has been created: the consent specification on the left and right, and the confirmation with the consent identifier below.

5.9 Prototype Evaluation and Discussion

Objective one involves the representation of patient consent as an evaluable authorisation object. The results of the validation carried out for this objective are shown in Figure 5.8 and Table 5.4, and the evaluation of the architecture against its security properties is shown in Table 5.5. RO1 is validated in two ways. First, the creation of the consent object is checked functionally: a valid specification must produce a consent object with all six components, and an invalid specification must be rejected before anything is stored. Second, the architecture is evaluated against the security properties it is designed to provide. The validation of the predicates that are evaluated against the consent object, VI, VA, VP, VT and VL, belongs to the chapters of RO2 to RO5, and the evaluation of the complete decision belongs to Chapter 10.

5.9.1 Validation of Consent Creation

Live scenarios. In the demonstration of 24 September 2026, Scenario 1 created the consent f050c29a for a patient with two guardians, did:parent:A and did:parent:B, the authorised doctor did:hospital:001, the four data modules, the purpose Treatment and a validity window ending on 31 December 2026. The audit ledger holds the CREATE_CONSENT event for this consent at 09:50:49 with the result SUCCESS, followed by a second consent, 04da25f5, created at 10:55:49 for another patient. Each creation was followed by the guardian approvals and the access decisions reported in the later chapters, which shows that the created object carried every component that the later checks needed.

Rejected specifications. In the local run of 11 October 2026, three invalid specifications were submitted before the valid one. A doctor DID that does not hold a valid doctor credential was rejected, a guardian DID that does not hold a valid guardian credential was rejected, and a specification that named the same guardian twice was rejected with the message shown in Figure 5.8. Nothing was stored and nothing was logged for the three rejected submissions. The fourth submission, with two distinct guardians holding valid credentials, created the consent 18ab9b62 at 17:41:36, which was then used for the access decisions reported in Chapters 6 to 9.

Automated tests. Two tests cover consent creation. The test test_patient_form_checks_the_doctor_and_the_guardians drives the patient form itself: it submits a malformed doctor DID, a guardian without a credential and a duplicate guardian, and asserts that each is refused with the right message, then submits a valid specification and asserts that the consent is created and logged. The test test_consent_persists_across_reload stores a consent, adds an approval, reloads the store from disk and asserts that the stored object is unchanged. The adversarial test A5 (test_A5_empty_scope_is_rejected) belongs to RO4 but also exercises the creation rule that the data scope must not be empty. All of these tests pass in the suite of 46 reported in Chapter 10.

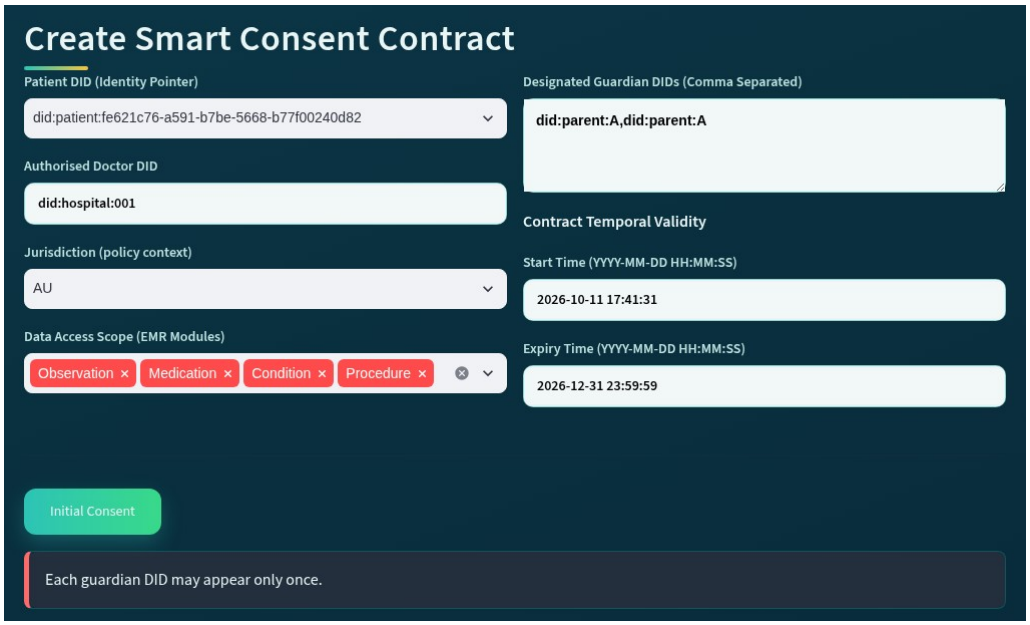


Figure 5.8: A specification that names the same guardian twice is rejected by Algorithm 1 before any consent is stored.

Table 5.4 brings these results together by component of the consent object. For each component of C it gives the check that Algorithm 1 applies at creation, the evidence from the live runs and the automated tests, and the result observed.

Table 5.4: Evaluation of the trust-centric consent model: the check applied to each component of C at creation, the evidence and the result

Component	**Check at creation (Algorithm 1)**	**Evidence**	**Result**
I Identity	Steps 1 to 3: the patient identity, the authorised doctor DID holding a valid doctor credential, and each guardian DID holding a valid guardian credential.	Run of 11 October 2026: a doctor DID without a valid doctor credential and a guardian DID without a valid guardian credential were rejected. Test test_patient_form_checks_the_doctor_and_the_guardians.	Invalid identities rejected before anything is stored; valid identities accepted.
A Authorisation	Step 3: the guardian set G and the threshold N; no guardian may appear twice.	A specification naming the same guardian twice was rejected (Figure 5.8). Scenario 1 of 24 September 2026 created consent f050c29a with the two guardians did:parent:A and did:parent:B.	Duplicate guardian rejected; consent created with N set to the number of guardians.
P Policy	Steps 4 and 5: the purpose and the data scope; the data scope must not be empty.	Adversarial test A5 (test_A5_empty_scope_is_rejected). Scenario 1 recorded the four data modules and the purpose Treatment; the purpose vocabulary check is exercised by the automated tests.	Empty scope rejected; purpose and scope recorded in the consent object.
T Time	Step 6: the validity interval T = (ts, te); the expiry time must be later than the start time.	Scenario 1 recorded a validity window ending on 31 December 2026; the prototype refuses an expiry time that is not later than the start time (Section 5.8).	Validity interval recorded; the interval is enforced at every request by VT (Chapter 9).

Component	Check at creation (Algorithm 1)	Evidence	Result
L Lifecycle	Steps 8 and 9: the consent identifier is generated and the object is created in the state PENDING_SIGNATURE.	Consents f050c29a, 04da25f5 and 18ab9b62 were created in PENDING_SIGNATURE. Test test_consent_persists_across_reload: the stored object, including its state, is unchanged after a reload from disk.	Created in PENDING_SIGNATURE; no access is possible until the state is ACTIVE (Chapter 9).
E Evidence	Step 11: the CREATE_CONSENT event is written to the audit ledger with the consent identifier, participants and timestamp.	Ledger events at 09:50:49 (f050c29a), 10:55:49 (04da25f5) and 17:41:36 (18ab9b62), each with the result SUCCESS. Nothing was logged for the three rejected submissions.	Every creation is logged; a rejected specification stores and logs nothing.

5.9.2 Evaluation of the Architecture against the Security Properties

The proposed MVC architecture provides several foundational security properties for the current stage. These properties emerge from the interaction between the Model, View, and Controller rather than from any single technology.

First, the architecture provides identity-dependent participation. A participant cannot contribute to the authorisation process solely by interacting with the system interface. The corresponding identity and credential must satisfy the verification requirements. Second, the architecture provides authority separation. Possession of a valid identity does not automatically result in permission to access patient data. Identity verification provides evidence concerning who an entity is, whereas authority verification determines whether that entity may perform a particular role within the authorisation process. Third, the architecture provides collective authorisation integrity. Sensitive healthcare data access can be configured to require multiple independently verified participants. Therefore, a single participant cannot unilaterally activate access when the defined authorisation threshold has not been reached. Fourth, the architecture provides policy-bounded access. A valid requester with sufficient approval evidence cannot access healthcare information beyond the purpose or data scope defined by the consent object. Finally, the architecture provides decision accountability. The system maintains evidence concerning the conditions that produced an authorisation decision, thereby supporting auditing and subsequent verification. These properties, together with the separation of interaction and enforcement established in Section 5.3, are summarised in Table 5.5, in which collective authorisation integrity and decision accountability appear as multi-party authorisation integrity and authorisation evidence respectively.

Table 5.5: Security properties supported by the proposed MVC-based consent architecture

Security property	Architectural mechanism	Security significance
Identity-dependent participation	DID and credential verification before a participant can contribute to the authorisation process	Designed to prevent unverified entities from being treated as legitimate participants.
Authority separation	Independent evaluation of identity and role or authority evidence	Designed to prevent successful authentication from being interpreted as unrestricted authorisation.
Multi-party authorisation integrity	Threshold evaluation over independently verified approval participants	Designed to prevent a single participant from unilaterally activating sensitive healthcare data access when collective approval is required.
Policy-bounded access	Evaluation of purpose and requested data scope against consent policy	Designed to prevent otherwise valid identities and approvals from being used to obtain data beyond the authorised scope.

Security property	Architectural mechanism	Security significance
Authorisation evidence	Recording of verification results, approval evidence, policy evaluation, and final decisions	Designed to provide traceability and support accountability, auditing, and future verification.
Separation of interaction and enforcement	The View submits requests while the Controller independently determines the security decision	Designed to prevent user-interface actions from being interpreted directly as trusted authorisation outcomes.

Each property in Table 5.5 is realised by a component of the architecture and tested in a later chapter. Identity-dependent participation and authority separation are tested in Chapter 6 through the credential checks and the forged-credential attack. Multi-party authorisation integrity is tested in Chapter 7 through the outside-signer, replayed-signature and duplicate-approval attacks. Policy-bounded access is tested in Chapter 8 through the over-scope and empty-scope attacks. Authorisation evidence is checked in every chapter through the audit ledger and in Chapter 10 through the recorded decisions. Separation of interaction and enforcement is the property that the hardening of 1 October 2026 made real: three attacks that the interface already blocked were also accepted by the service functions until the checks were moved into the Controller, as reported in Chapter 10. The architecture therefore does not claim security from MVC itself; it claims it from the placement of every trust-sensitive check inside the Controller, and the tests of the following chapters are the evidence for that claim.

5.9.3 Discussion

The use of the consent object for consent creation has several advantages which include completeness, rejection before storage, and evidence. Completeness means that a created consent carries every condition that the later checks need, so no check has to look for its condition outside the object. Rejection before storage means that an invalid specification leaves no record behind, so the store never holds a consent that cannot be evaluated. Evidence means that every creation is written to the audit ledger with its identifier, its participants and its time, so the later decisions can be traced back to the consent they were taken on.

On the other hand, the trade-off is rigidity. A consent cannot be changed after it has been created, so a change of doctor, guardian or data scope requires a new consent, and the credentials of the doctor and of every guardian must have been issued before the consent can be created (see Chapter 6). In the current interface, the purpose is also fixed to Treatment. Although these may be considered limitations of the method, the certainty that every stored consent is complete and that every creation is recorded outweighs them at this stage, and consent modification is addressed in the next stage of the research (Chapter 9).

5.10 Conclusion

In this chapter, we discussed the research architecture on which the whole framework is built. In particular, we discussed the three layers of the architecture, where trust is evaluated and where the boundaries between the layers lie, and how the architecture maps to the six research objectives. In addition, we proposed a model to represent patient consent as the evaluable object C = (I, A, P, T, L, E), created through Algorithm 1 only when every field is valid, assigned a unique identifier, persisted in the state PENDING_SIGNATURE and recorded in the audit ledger. This addressed the first research objective of this thesis.

In this chapter we also presented the prototype used for validating the solution to research objective 1. This was done using a public synthetic dataset. At the end of the validation process, the results show that valid specifications are created with all six

components and that invalid ones are rejected before anything is stored, and the architecture was evaluated against the six security properties it is designed to provide. However, a key finding from the validation was that a consent cannot be changed once it has been created; the users of HALAH will have to create a new consent when its terms change.

The next chapter will discuss in a methodological manner the steps involved in verifying who the participants are and what authority they hold, which is the solution for research objective 2.